Overview Tasks Solutions

Not Running
For the best experience, choose the region closest to you. Then, start the lab to begin.
Select the region closest to you
US-East
Keyboard layout: US English (US)
Start lab

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Emulate and hunt for library injection using Osquery

First, let's access the container and start our *sample-target* process.

```
docker exec -it target-machine bash
cd /root/Tools/linux-inject
./sample-target
```

```
root@kali:~# docker exec -it target-machine bash
root@target-machine:~# cd /root/Tools/linux-inject
root@target-machine:~/Tools/linux-inject# ./sample-target
sleeping...
sleeping...
sleeping...
█
```

Leave this session running and open a new container session and perform the injection as follows.

```
docker exec -it target-machine bash
cd /root/Tools/linux-inject
./inject -n sample-target sample-library.so
```

```
root@kali:~# docker exec -it target-machine bash
root@target-machine:~# cd /root/Tools/linux-inject
root@target-machine:~/Tools/linux-inject# ./inject -n sample-target sample-library.so
targeting process "sample-target" with pid 52
"sample-library.so" successfully injected
root@target-machine:~/Tools/linux-inject# █
```

The injection performed successfully.

This is what you will see inside the SSH session where *sample-target* is running.

```
sleeping...
sleeping...
sleeping...
sleeping...
sleeping...
I just got loaded
sleeping...
sleeping...
sleeping...
sleeping...
```

Let's continue with the detection part, as follows.

```
osqueryi
select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';
```

```
root@target-machine:~/Tools/linux-inject# osqueryi
Using a virtual database. Need help, type '.help'
osquery> select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';
+--------------------------+-----+-----------+---------------+--------------+--------+
| name                     | oid | subsystem | current_value | config_value | type   |
+--------------------------+-----+-----------+---------------+--------------+--------+
| kernel.yama.ptrace_scope |     | kernel    | 0             | 1            | string |
+--------------------------+-----+-----------+---------------+--------------+--------+
osquery> █
```

Still inside the osquery "shell", execute the below.

```
SELECT process_memory_map.*, pid as mpid from process_memory_map LEFT JOIN processes
```

```
osquery> SELECT process_memory_map.*, pid as mpid from process_memory_map LEFT JOIN processes USING (pid) WHERE process_memory_map.path LIKE '/%' and process
_memory_map.pseudo != 1 AND process_memory_map.path NOT LIKE '/lib/%' AND process_memory_map.path NOT LIKE '/usr/lib%' AND process_memory_map.path != process
es.path AND process_memory_map.permissions LIKE '%x%';
+-----+----------------+----------------+-------------+--------+--------+---------+-----------------------------------------+--------+------+
| pid | start          | end            | permissions | offset | device | inode   | path                                    | pseudo | mpid |
+-----+----------------+----------------+-------------+--------+--------+---------+-----------------------------------------+--------+------+
| 52  | 0x7f3bfc5cf000 | 0x7f3bfc5d0000 | r-xp        | 0      | ca:01  | 1082770 | /root/Tools/linux-inject/sample-library.so | 0   | 52   |
+-----+----------------+----------------+-------------+--------+--------+---------+-----------------------------------------+--------+------+
osquery> █
```

The above query looks at the process memory using Osquery's *process_memory_map* table. We are interested in memory locations that have the execute permission. The original image as well as locations marked as pseudo should be disregarded. We should also remove known paths where libraries are loaded. Now, delete the injected library (you can exit osqueryi by executing *.exit*).

```
CTRL + C
ls
rm sample-library.so
```

```
osquery> ^C

root@target-machine:~/Tools/linux-inject# ls
LICENSE.txt  README.md  inject-arm.c  inject-x86_64.c  ptrace.c  sample-library.c   sample-library32.so  sample-target.c  slides_BHArsenal2015.pdf  utils.h
root@target-machine:~/Tools/linux-inject# █
```

Execute *osqueryi* again and issue the query below.

```
osqueryi
SELECT process_memory_map.pid, process_memory_map.start, process_memory_map.end, proce
```

```
root@target-machine:~/Tools/linux-inject# osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT process_memory_map.pid, process_memory_map.start, process_memory_map.end, process_memory_map.permissio
memory_map.path from process_memory_map LEFT join file USING (path) where pseudo != 1 AND process_memory_map.path NOT
th NOT LIKE '/usr/lib%' AND process_memory_map.permissions LIKE '%x%' AND filename IS NULL and process_memory_map.inod
s = 'r-xp';
+-----+----------------+----------------+-------------+--------+--------------------------------------------+
| pid | start          | end            | permissions | offset | path                                       |
+-----+----------------+----------------+-------------+--------+--------------------------------------------+
| 52  | 0x7f3bfc5cf000 | 0x7f3bfc5d0000 | r-xp        | 0      | /root/Tools/linux-inject/sample-library.so |
+-----+----------------+----------------+-------------+--------+--------------------------------------------+
osquery> █
```

The injected library is still visible! Terminate osqueryi. Let's try the other injection tool. Terminate *sample-target* and start it again inside the other SSH session.

```
cd /root/Tools/ReflectiveSOInjection/inject/src
ps aux (to identify the PID of sample-target)
```

```
root@target-machine:~/Tools/linux-inject# cd /root/Tools/ReflectiveSOInjection/inject/src
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 126920 18096 ?        SNsl 04:50   0:00 osqueryd --flagfile=/etc/osquery/osquery.flags --tls_hostname=10.0.5.10:8412
root           8  0.1  0.3 1180616 27824 ?       SNl  04:50   0:01 osqueryd
root          42  0.0  0.0  18512  3388 pts/0    Ss   04:57   0:00 bash
root          52  0.0  0.0   6568  1500 pts/0    S+   04:59   0:00 ./sample-target
root          53  0.0  0.0  18512  3420 pts/1    Ss   04:59   0:00 bash
root          75  0.0  0.0  34404  2796 pts/1    R+   05:06   0:00 ps aux
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# ./inject -p 52 ../../so/src/ReflectiveLoader.so
[i] targeting process with pid 52
[+] shared object mapped at 0x7f49ad298000
[+] found dynamic segment at 0x7f49ad29be20
[+] dynsym found at address 0x7f49ad298250
[+] dynstr found at address 0x7f49ad298418
[+] Resolved ReflectiveLoader offset to 0x7aa
[i] Setting target registers to appropriate values
[i] Overwriting target memory region with shellcode
[+] Transfering execution to stage 0 shellcode
[+] Returned from Stage 0 shell code RIP of target is 0x40006f
[i] Stage 0 mmap returned memory address of 0x7f3bfcdde000.. verifying allocation succeeded..
[+] Okay.. mmap allocation was successful!
[+] Writing our shared object into the victim process address space MUAHAHAHA!!!
[+] Setting RIP to ReflectiveLoader function
[+] Calling ReflectiveLoader function! Let's hope this works ;D
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# █
```

Let's detect the library injection, again using osquery.

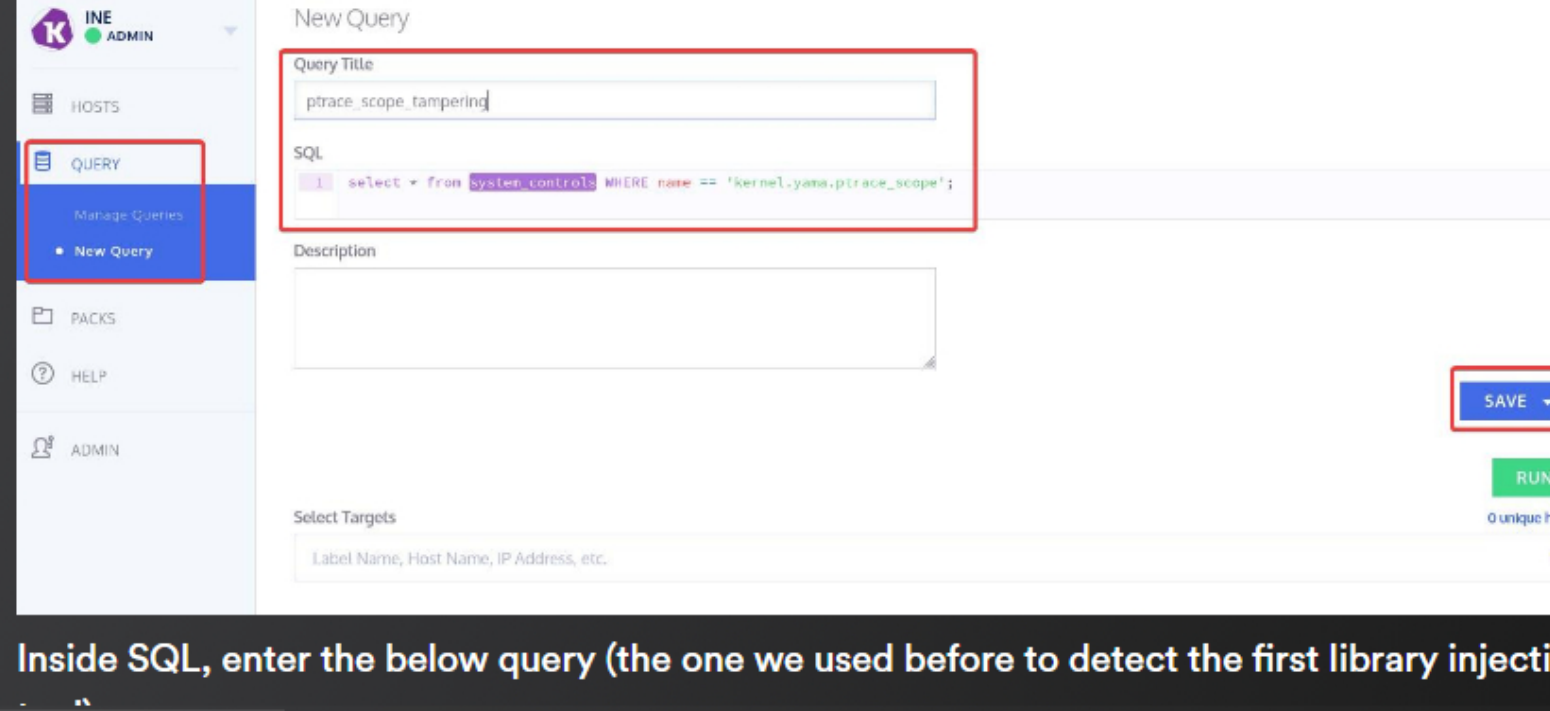
```
osqueryi
SELECT processes.name, process_memory_map.*, pid as mpid from process_memory_map join
```

```
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT processes.name, process_memory_map.*, pid as mpid from process_memory_map join processes USING (pid) WHERE
rwxp' AND process_memory_map.path = '';
+---------------+-----+----------------+----------------+-------------+--------+--------+-------+------+--------+------+
| name          | pid | start          | end            | permissions | offset | device | inode | path | pseudo | mpid |
+---------------+-----+----------------+----------------+-------------+--------+--------+-------+------+--------+------+
| sample-target | 52  | 0x7f3bfbfc9000 | 0x7f3bfc3cb000 | rwxp        | 0      | 00:00  | 0     |      | 0      | 52   |
| sample-target | 52  | 0x7f3bfcdde000 | 0x7f3bfcde5000 | rwxp        | 0      | 00:00  | 0     |      | 0      | 52   |
+---------------+-----+----------------+----------------+-------------+--------+--------+-------+------+--------+------+
osquery> █
```

The advantage of the second library injection technique is that the injected shared object doesn't have to be on disk. That's why the query above searches for memory regions marked as rwxp having an empty path.

Task 2. Hunt for library injection using at scale using Kolide Fleet

First, log into Kolide Fleet and create a new query, as follows.

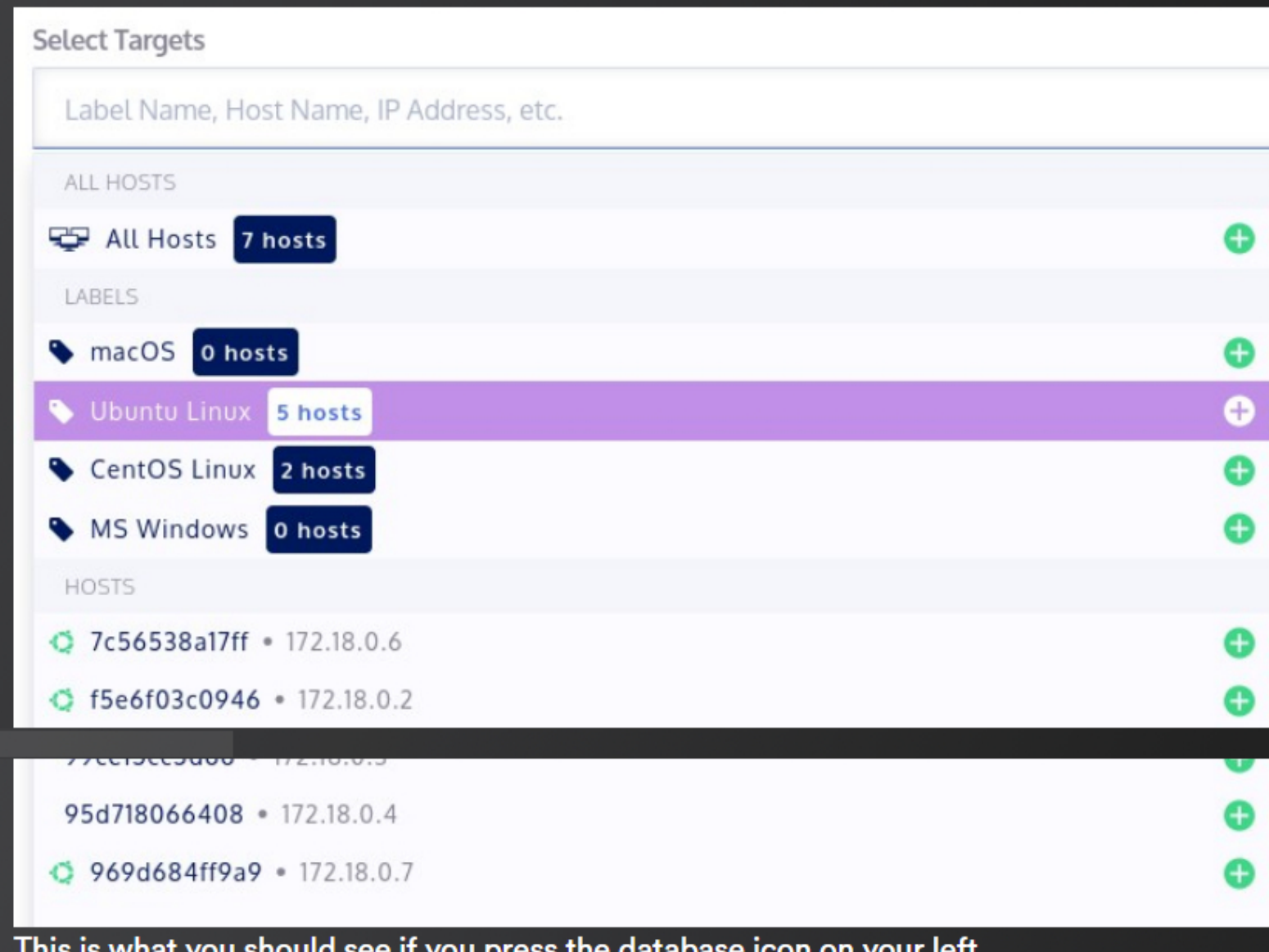

INE ADMIN
HOSTS
QUERY
Manage Queries
New Query
PACKS
HELP
ADMIN
New Query
Description
SAVE
RUN
0 unique hosts
Select Targets
Label Name, Host Name, IP Address, etc.

Query Title: ptrace_scope_tampering
SQL: `select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';`

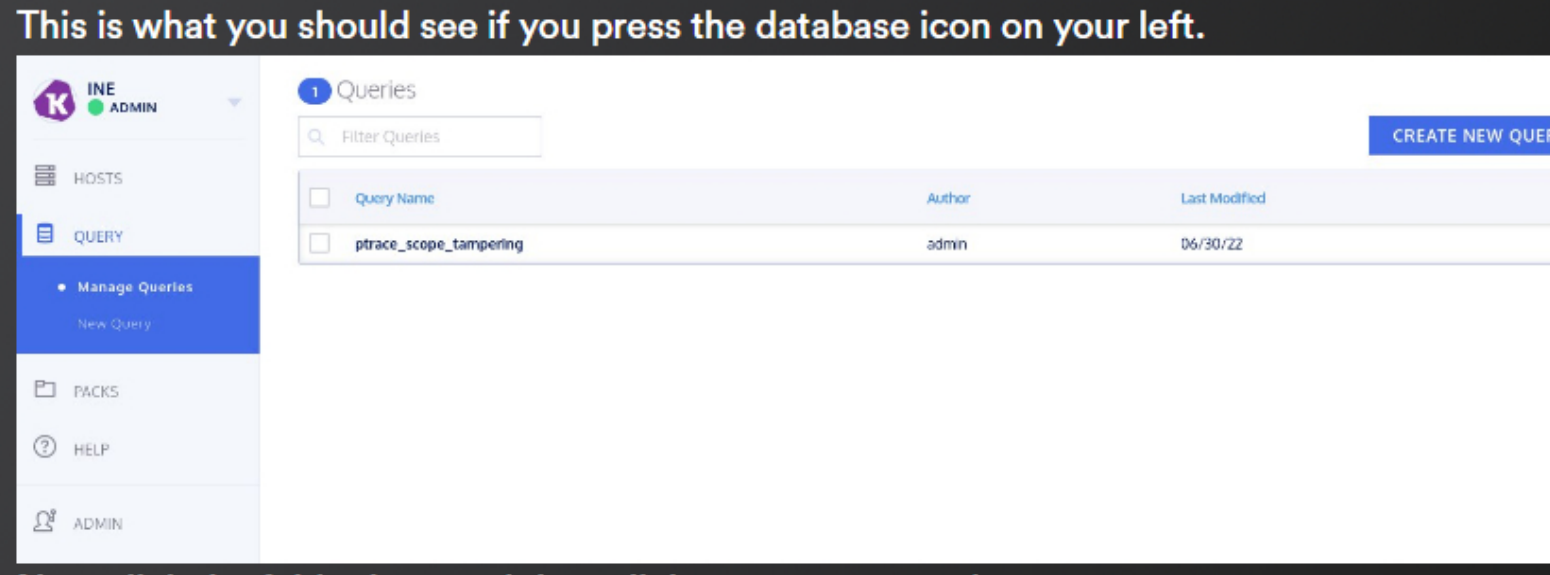
Inside SQL, enter the below query (the one we used before to detect the first library injection

```
select * from system_controls WHERE name == 'kernel.yama.ptrace_scope';
```

Then, select all Ubuntu hosts, by pressing the + sign next to them and finally click SAVE (as new)

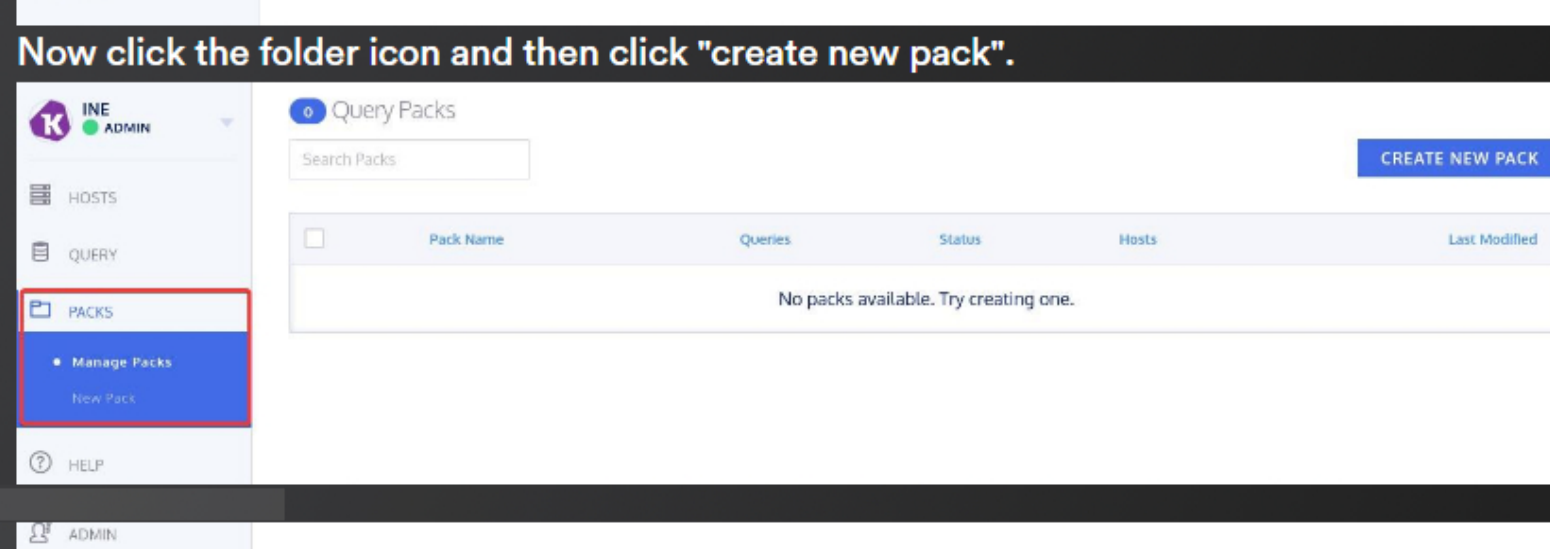

Select Targets
Label Name, Host Name, IP Address, etc.
ALL HOSTS
All Hosts 7 hosts
LABELS
macOS 0 hosts
Ubuntu Linux 5 hosts
CentOS Linux 2 hosts
MS Windows 0 hosts
HOSTS
7c56538a17ff • 172.18.0.6
f5e6f03c0946 • 172.18.0.2
95d718066408 • 172.18.0.4
969d684ff9a9 • 172.18.0.7

This is what you should see if you press the database icon on your left.


INE ADMIN
HOSTS
QUERY
Manage Queries
New Query
PACKS
HELP
ADMIN
1 Queries
Filter Queries
CREATE NEW QUERY

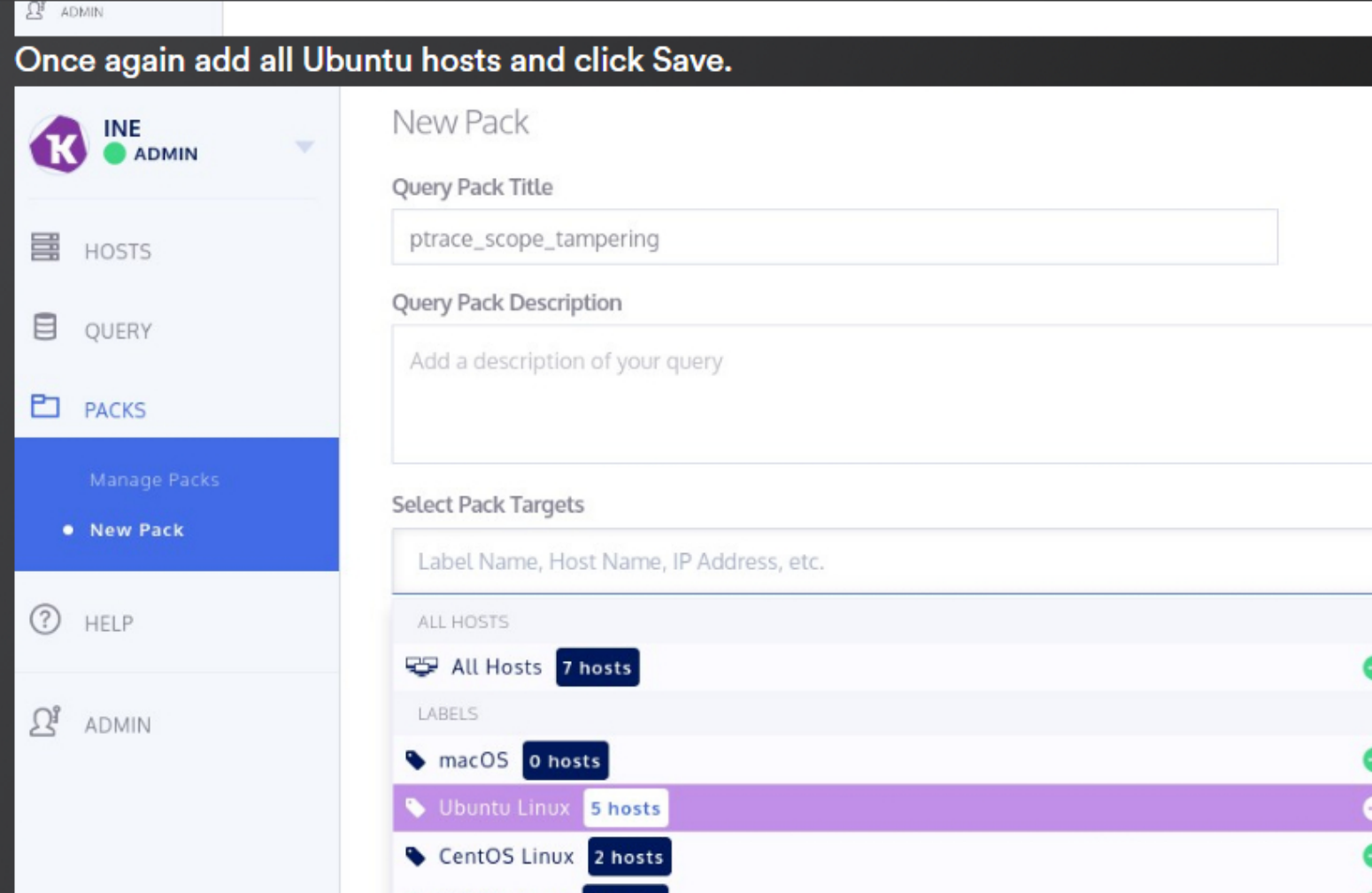
Query Name	Author	Last Modified
ptrace_scope_tampering	admin	06/30/22

Now click the folder icon and then click "create new pack".


INE ADMIN
HOSTS
QUERY
PACKS
Manage Packs
New Pack
HELP
ADMIN
0 Query Packs
Search Packs
CREATE NEW PACK
Pack Name
Queries
Status
Hosts
Last Modified

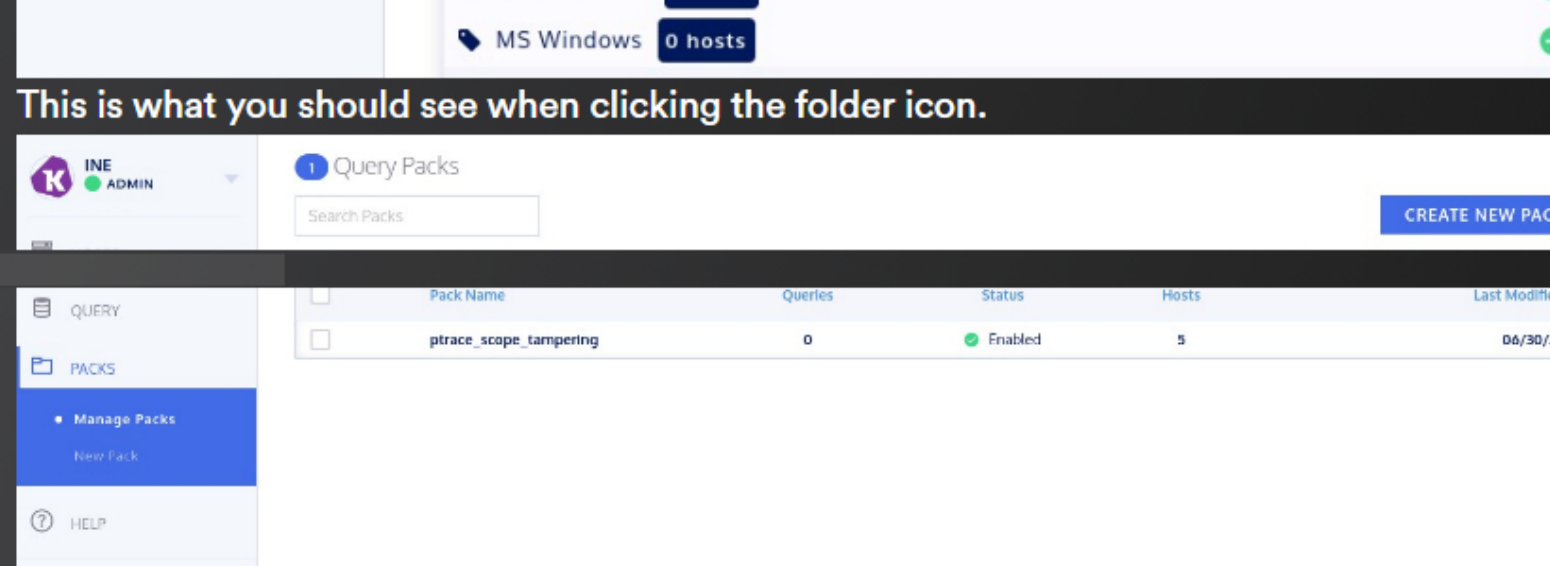
No packs available. Try creating one.

Once again add all Ubuntu hosts and click Save.


INE ADMIN
HOSTS
QUERY
PACKS
Manage Packs
New Pack
HELP
ADMIN
New Pack
Query Pack Description
Add a description of your query
Select Pack Targets
Label Name, Host Name, IP Address, etc.
ALL HOSTS
All Hosts 7 hosts
LABELS
macOS 0 hosts
Ubuntu Linux 5 hosts
CentOS Linux 2 hosts
MS Windows 0 hosts

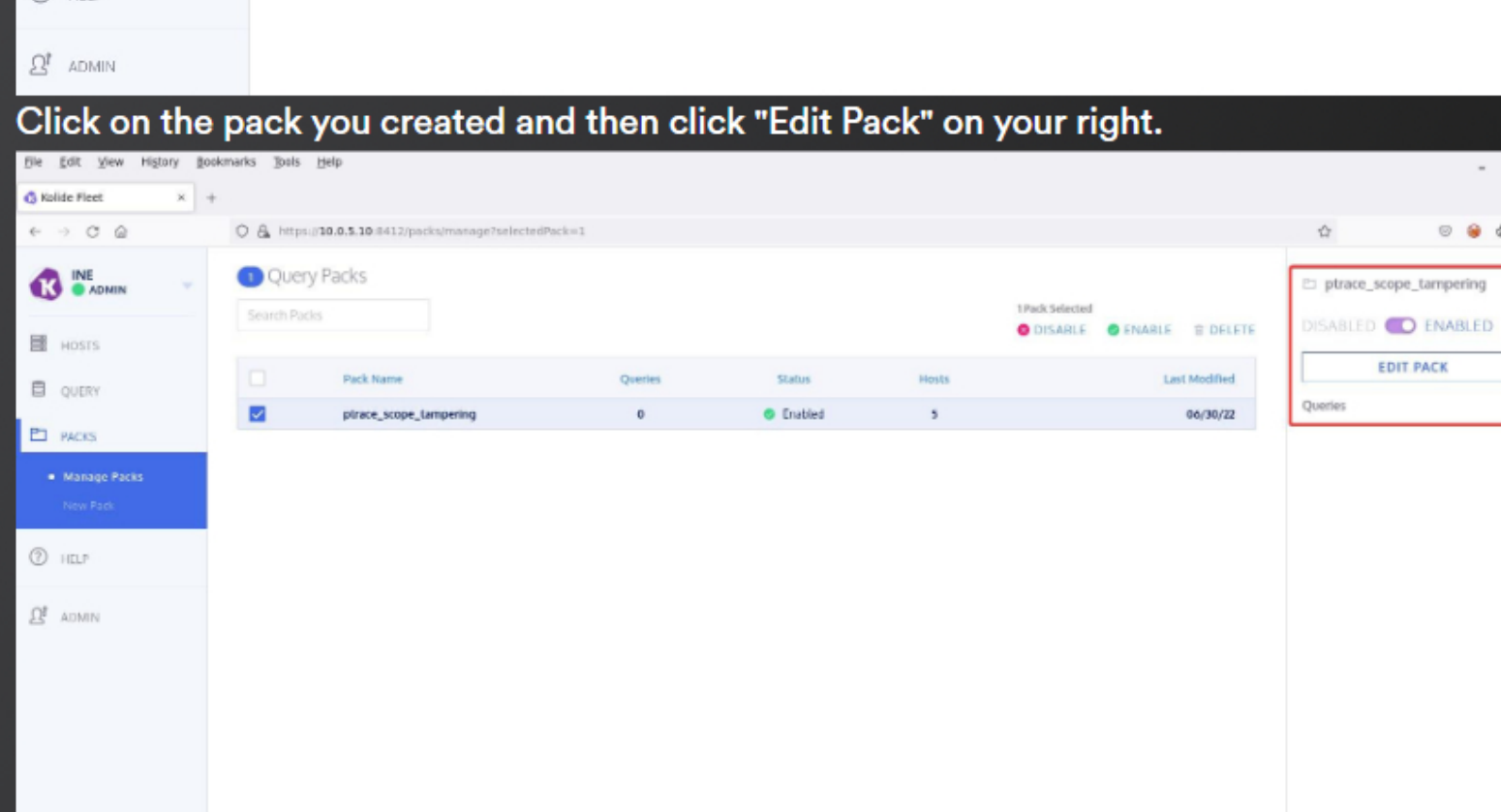
Query Pack Title: ptrace_scope_tampering

This is what you should see when clicking the folder icon.

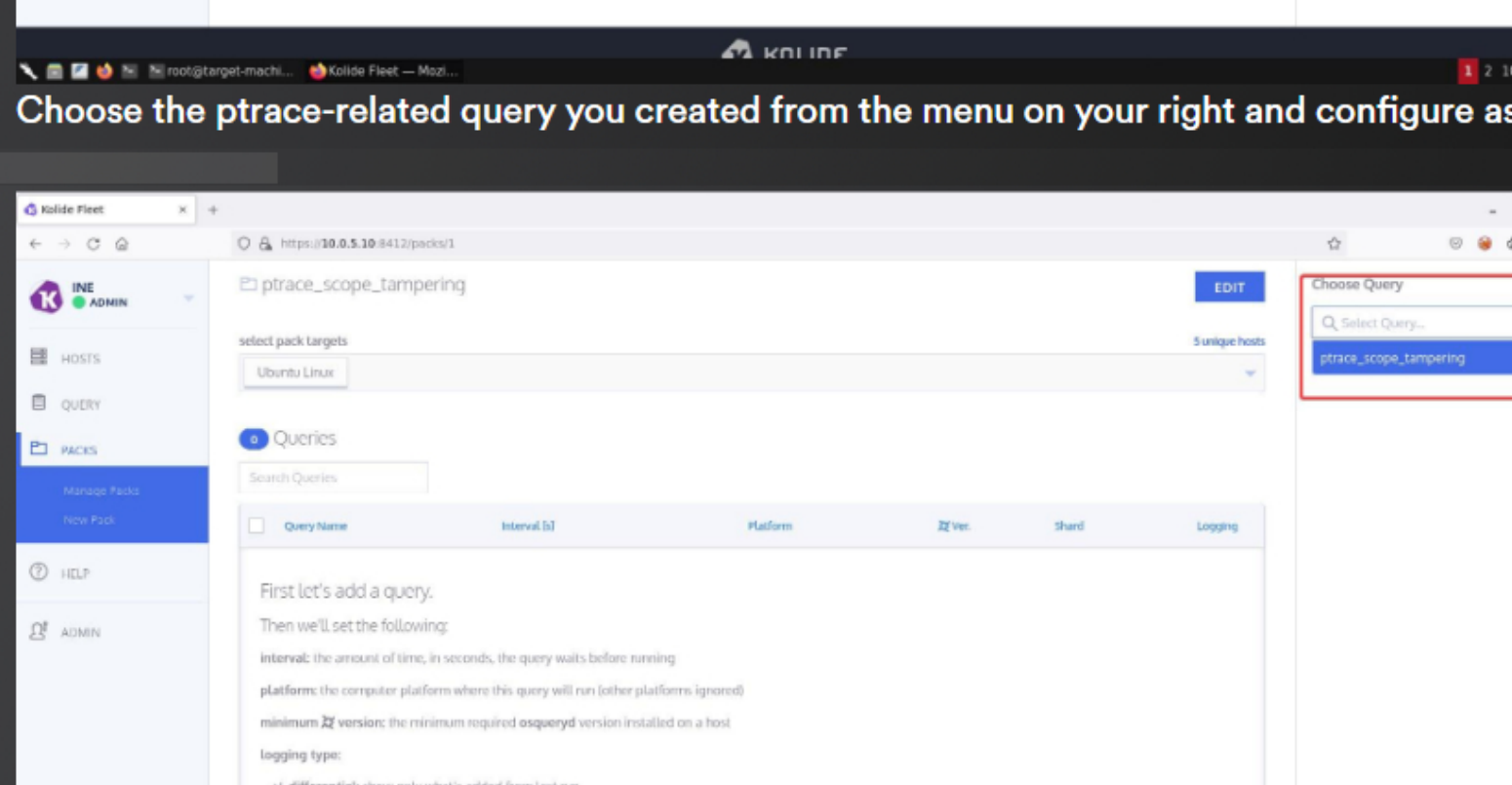

1 Query Packs
Search Packs
CREATE NEW PACK

Pack Name	Queries	Status	Hosts	Last Modified
ptrace_scope_tampering	0	Enabled	5	06/30/22

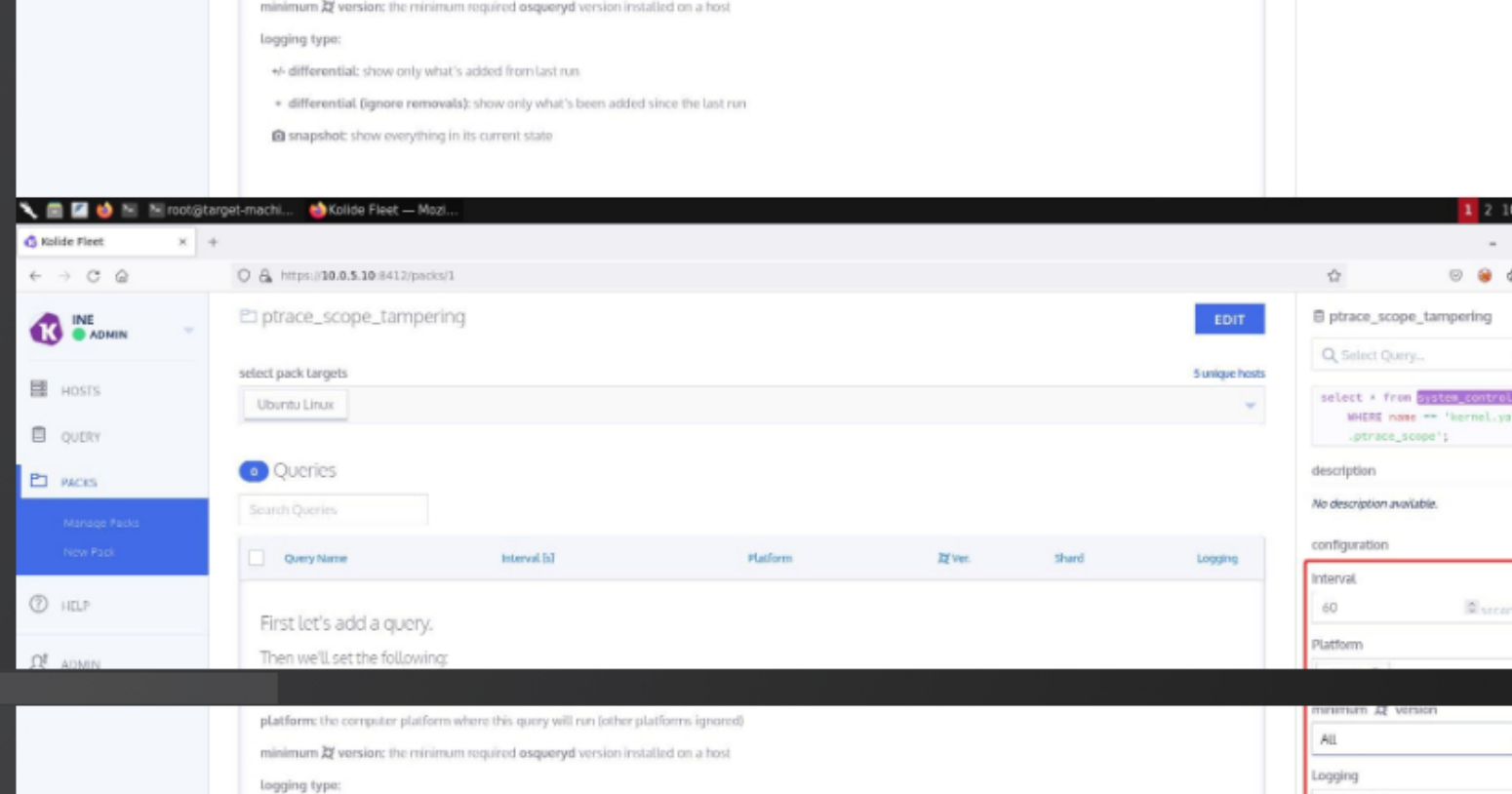
Click on the pack you created and then click "Edit Pack" on your right.


Kolide Fleet
Query Packs
ptrace_scope_tampering
DISABLED ENABLED
EDIT PACK
Queries

Choose the ptrace-related query you created from the menu on your right and configure as

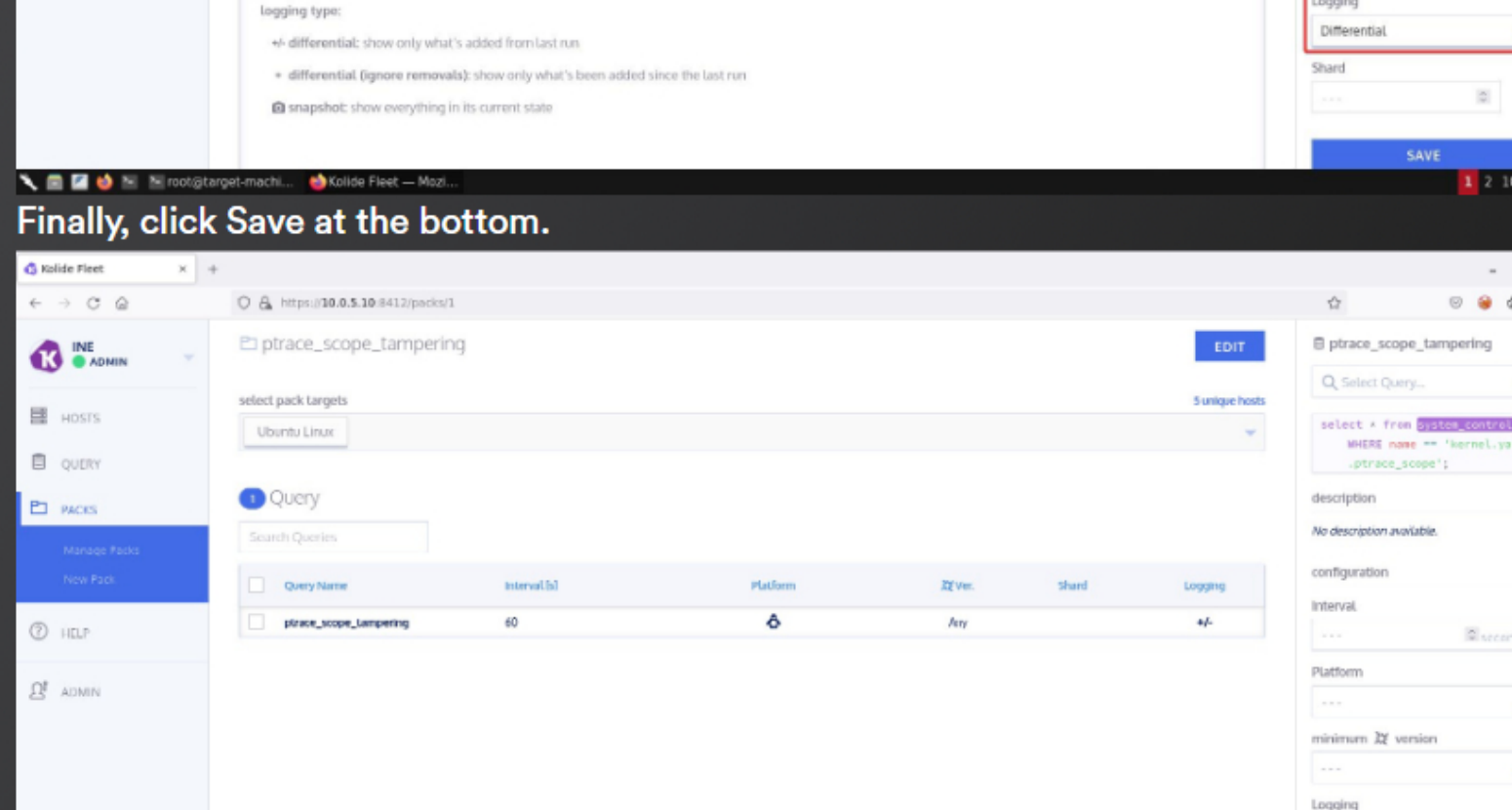

ptrace_scope_tampering
EDIT
Choose Query
Select Query...
ptrace_scope_tampering
Queries

First let's add a query.
Then we'll set the following:
interval: the amount of time, in seconds, the query waits before running
platform: the computer platform where this query will run (other platforms ignored)
minimum version: the minimum required osqueryd version installed on a host
logging type:
differential: show only what's added from last run
differential (ignore removals): show only what's been added since the last run
snapshot: show everything in its current state


ptrace_scope_tampering
description
No description available.
configuration
SAVE

Interval: 60
Platform: All
Logging: Differential

Finally, click Save at the bottom.


ptrace_scope_tampering
EDIT
1 Query
SAVE

Query Name	Interval (s)	Platform	Ver.	Shard	Logging
ptrace_scope_tampering	60		Any		

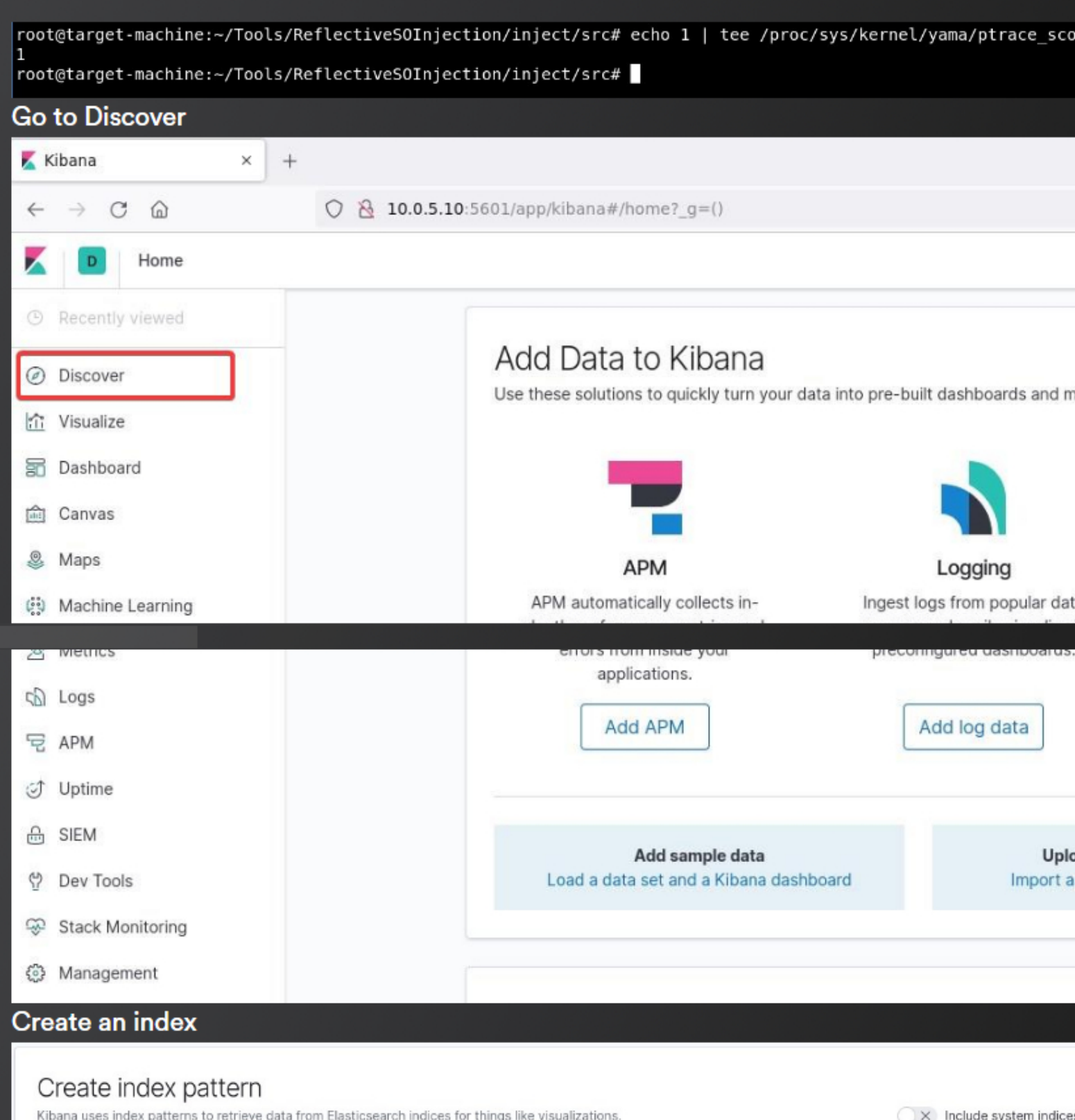
Fleet logs are being pushed to an ELK instance at **http://10.0.5.10:5601**

There you should be able to identify changes to the ptrace scope, at scale. For example, change back the ptrace scope on the *osquery_client* machine as follows and you should see a new event inside Kibana.

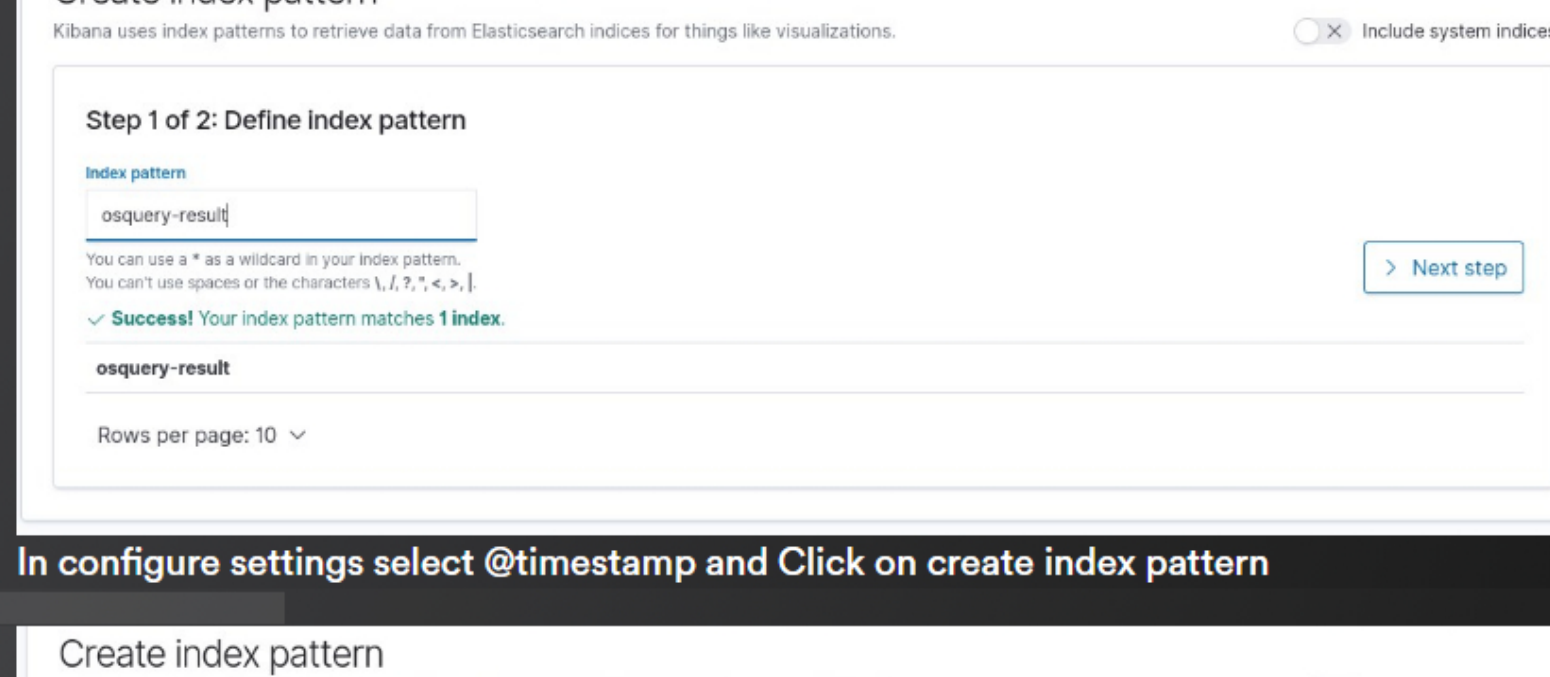
```
echo 1 | tee /proc/sys/kernel/yama/ptrace_scope
```

```
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# echo 1 | tee /proc/sys/kernel/yama/ptrace_scope
1
root@target-machine:~/Tools/ReflectiveSOInjection/inject/src# █
```

Go to Discover

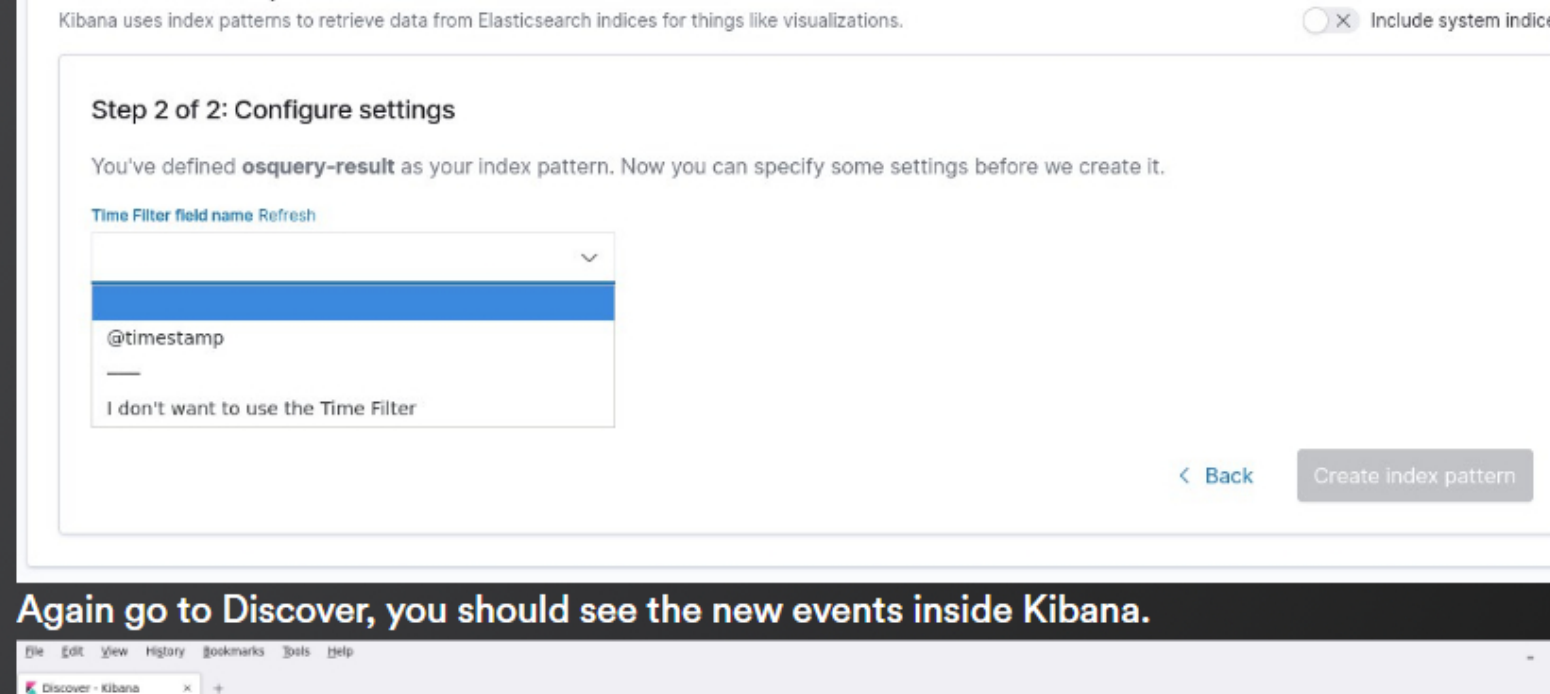

Kibana
10.0.5.10:5601/app/kibana#/home?_g=()
Home
Recently viewed
Discover
Visualize
Dashboard
Canvas
Maps
Machine Learning
Logs
APM
Uptime
SIEM
Dev Tools
Stack Monitoring
Management
Add Data to Kibana
Use these solutions to quickly turn your data into pre-built dashboards and monit
APM
APM automatically collects in-
applications.
Logging
Ingest logs from popular data
Add APM
Add log data
Add sample data
Load a data set and a Kibana dashboard
Upload d
Import a CSV

Create an index



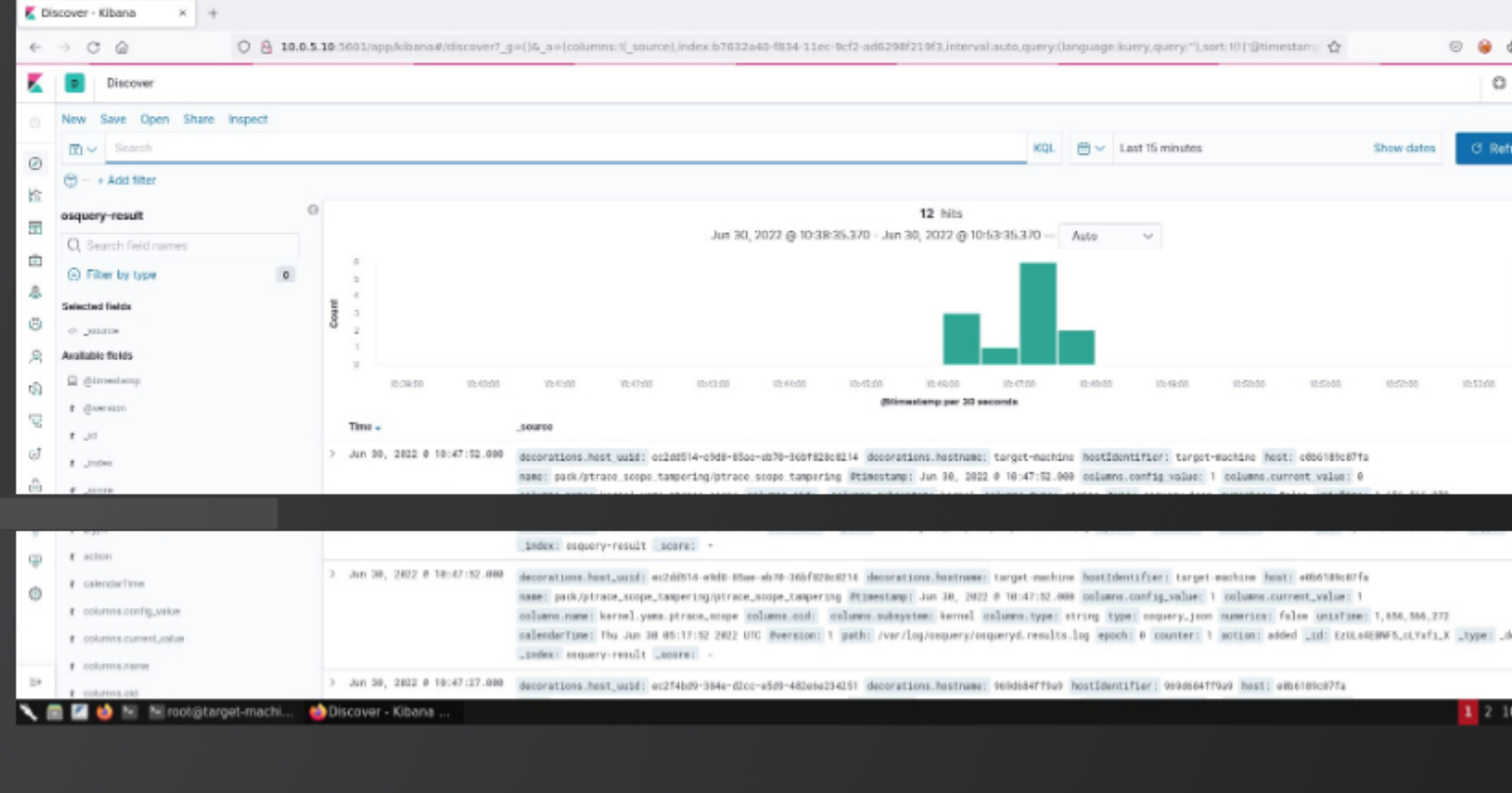
Create index pattern
Kibana uses index patterns to retrieve data from Elasticsearch indices for things like visualizations.
Include system indices
Step 1 of 2: Define index pattern
Index pattern: osquery-result
You can use a * as a wildcard in your index pattern.
You can't use spaces or the characters \, /, ?, ", <, >, |.
✓ Success! Your index pattern matches 1 index.
osquery-result
Rows per page: 10
> Next step

In configure settings select @timestamp and Click on create index pattern



Create index pattern
Kibana uses index patterns to retrieve data from Elasticsearch indices for things like visualizations.
Include system indices
Step 2 of 2: Configure settings
You've defined **osquery-result** as your index pattern. Now you can specify some settings before we create it.
Time Filter field name Refresh
@timestamp
I don't want to use the Time Filter
< Back
Create index pattern

Again go to Discover, you should see the new events inside Kibana.


Discover - Kibana
Discover
New Save Open Share Inspect
KQL
Last 15 minutes
Refresh
osquery-result
12 hits
Count
@timestamp per 30 seconds

Resources

1. https://www.trustedsec.com/blog/linux-hows-my-memory/
2. https://cybersecurity.att.com/blogs/labs-research/hunting-for-linux-library-injection-with-osquery

KEY TAKEWAYS

1. There are numerous ways for an adversary to achieve their goal. Being familiar with as many as possible attacks can aid hunters in creating better detection rules performing successful hunts.
2. Hunting playgrounds are a great source of different attacks, helping hunters in increasing their abilities.